Computer Programming 1

Lecture 3 and 4 – Representations and Types (part 1)

- Adolfo Villafiorita
- adolfo@shair.tech

Outline

- Representations *(part 1)*
  - Encoding: binary, decimal, hexadecimal
  - Conversions
  - Operations
- Fundamental types
  - Boolean
  - Integer
  - Real
  - Enum
  - Char

Binary representation

- The binary representation of numbers takes place via sequences of bits (ones and zeros).
- We distinguish the representation by:
  - positive integers
  - signed integers
  - real numbers

Positive integer numbers

- A sequence of bits $b_{n-1} \ldots b_1 \, b_0$ represents the number:

  Where: 1 + 1 = 10

  - E.g., (with 8 bits)

    00001001 represents the number 9 ($2^3 + 2^0 = 8 + 1 = 9$)

    $00001001 = 1000 + 0001 = 2^3 + 2^0$

    $00001001 = 8 + 1 = 9$

- Given *n* bits, it is possible to represents the numbers in the range $[0, 2^n-1]$
  - E.g., (with 8 bits) the range [0, 255] (i.e., $2^8-1=255$)
  - E.g., (with 32 bits) the range [0, 4294967295]

Decimal to Binary conversion

- A sequence of divisions per 2 and analysis of the remainder
- Convert 728_{10} in binary



Decimal to Binary conversion

- A sequence of divisions per 2 and analysis of the remainder
- Convert 728_{10} in binary

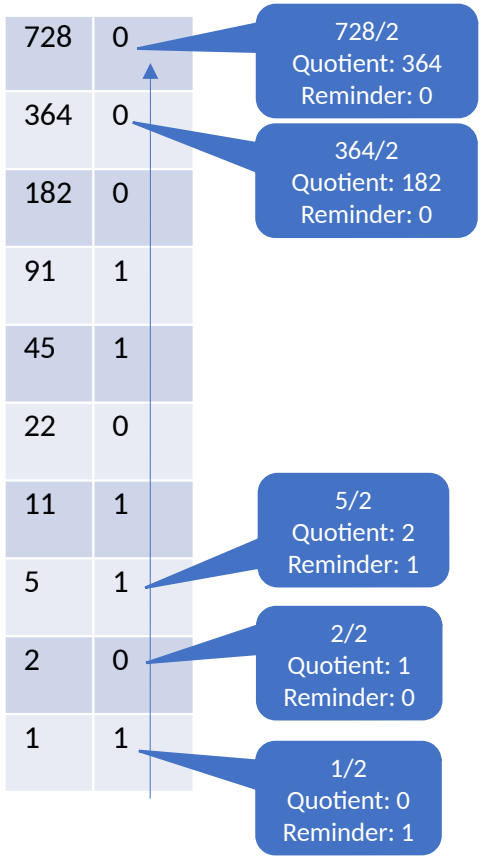
728	0
364	0

728/2
Quotient: 364
Reminder: 0

364/2
Quotient: 182
Reminder: 0

Decimal to Binary conversion

- A sequence of divisions per 2 and analysis of the remainder
- Convert 728_{10} in binary



Result: 1011011000_2

Decimal to Binary conversion

- A sequence of divisions per 2 and analysis of the remainder
- Convert 728_{10} in binary

728	0
364	0
182	0
91	1
45	1
22	0
11	1
5	1
2	0
1	1

728/2
Quotient: 364
Reminder: 0

364/2
Quotient: 182
Reminder: 0

5/2
Quotient: 2
Reminder: 1

2/2
Quotient: 1
Reminder: 0

1/2
Quotient: 0
Reminder: 1

Result: 1011011000_2

Test

$$2^9 + 2^7 + 2^6 + 2^4 + 2^3 = 512 + 128 + 64 + 16 + 8 = \sum_{i=0}^{n-1} 2^i * b_i$$

Decimal to Binary conversion (2)

- Convert 3249_{10} in binary

3249	1

Decimal to Binary conversion (2)

- Convert 3249_{10} in binary

3249	1
1624	0

Decimal to Binary conversion (2)

- Convert 3249_{10} in binary

3249	1
1624	0
812	0
406	0
203	1
101	1
50	0
25	1
12	0
6	0
3	1
1	1

Decimal to Binary conversion (2)

- Convert 3249_{10} in binary

3249	1
1624	0
812	0
406	0
203	1
101	1
50	0
25	1
12	0
6	0
3	1
1	1

Result: 110010110001_2

Decimal to Binary conversion (2)

- Convert 3249_{10} in binary

3249	1
1624	0
812	0
406	0
203	1
101	1
50	0
25	1
12	0
6	0
3	1
1	1

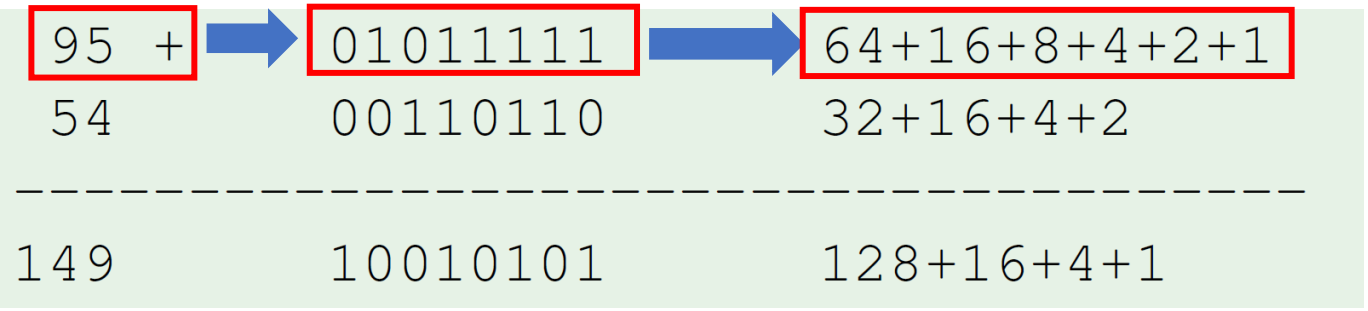
Result: 110010110001_2

$$\sum_{i=0}^{n-1} 2^i * b_i$$

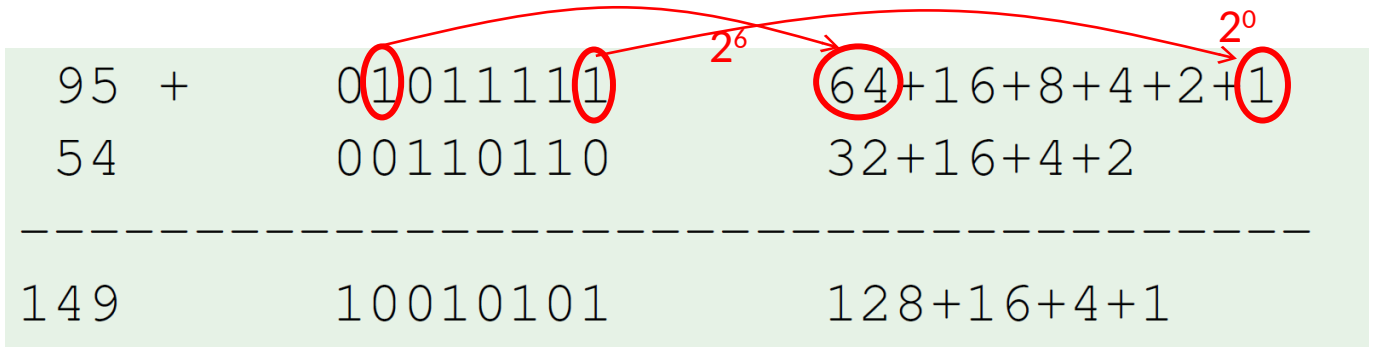
Test

$$2^{11} + 2^{10} + 2^7 + 2^5 + 2^4 + 2^0 = 2048 + 1024 + 128 + 32 + 16 + 1$$

Examples: operations with positive integers



Examples: operations with positive integers



Examples: operations with positive integers



95 +	01011111	64+16+8+4+2+1
54	00110110	32+16+4+2
149	10010101	128+16+4+1

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0 (1)
0 + 1 = 1
```

10
=
(1)0

0 with carry 1

Examples: operations with positive integers



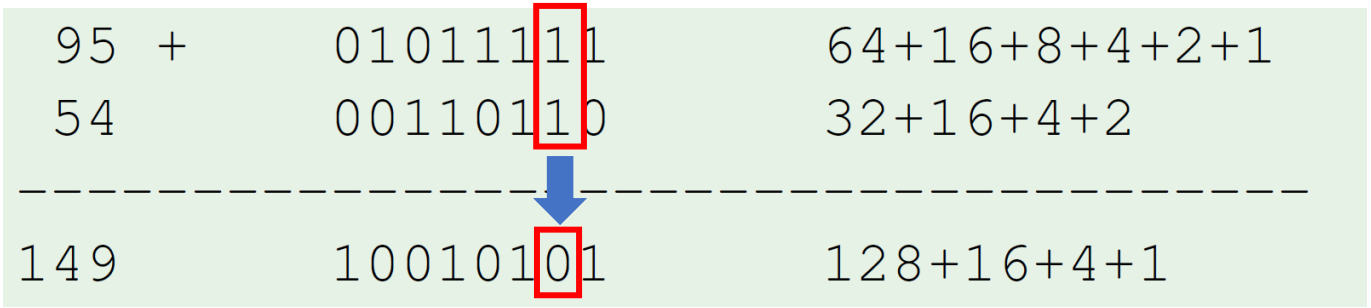
```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
------------------------------------------
149       10010101        128+16+4+1
```

1+0 = 1

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0 (1)
  0 + 1 = 1
```

10 = (1)0

Examples: operations with positive integers



```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------
149       10010101        128+16+4+1
```

1+1 = 0 (carry 1)

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0 (1)
  0 + 1 = 1
```

10 = (1)0

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------------
149       10010101        128+16+4+1
```

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0 (1)
  0 + 1 = 1
```

10 = (1)0

1+1 = 0 (carry 1) ->
0 (carry 1) + previous carry 1 = 1 (carry 1)

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------
149       10010101        128+16+4+1
```

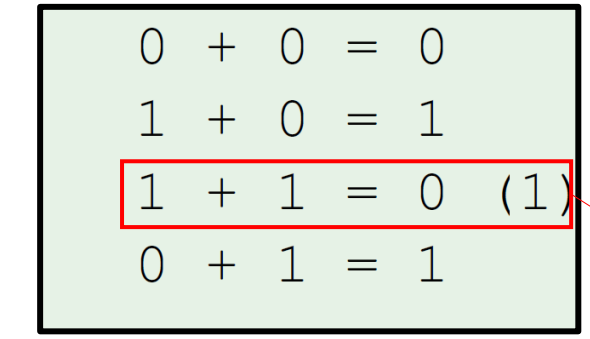
1+0 = 1 ->
1 + previous carry 1 = 0 (carry 1)

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0 (1)
  0 + 1 = 1
```

10
=
(1)0

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------
149       10010101        128+16+4+1
```

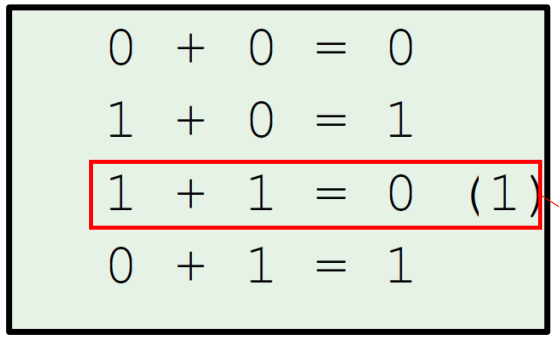


10
=
(1)0

1+1 = 0 (carry 1) ->
0 (carry 1) + previous carry 1 = 1 (carry 1)

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
------------------------------------------
149       10010101        128+16+4+1
```



10
=
(1)0

0+1 = 1->
1+ previous carry 1 = 0 (carry 1)

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
----------------------------------------
149       10010101        128+16+4+1
```

1+0 = 1->
1+ previous carry 1 = 0 (carry 1)

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0  (1)
  0 + 1 = 1
```

10
=
(1)0

Examples: operations with positive integers

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
----------------------------------------
149       10010101        128+16+4+1
```

0+0 = 0->
0 + previous carry 1 = 1

10010101 (binary) = 149 (decimal)

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0 (1)
  0 + 1 = 1
```

10 = (1)0

Examples: operations with positive integers

```
149 -     10010101      128+16+4+1
095       01011111      64+16+8+4+2+1
----------------------------------------
 54       00110110      32+16+4+2
```

```
  0 - 0 = 0
  1 - 0 = 1
  1 - 1 = 0
  0 - 1 = 1  (1)
```

11 = (1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
--------------------------------------
 54       00110110        32+16+4+2
```

1-1 = 0

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

11 = (1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
----------------------------------------
 54       00110110        32+16+4+2
```

0-1 = 1 (borrow 1)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

11
=
(1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
----------------------------------------
 54       00110110        32+16+4+2
```

1-1 = 0 ->
0 – previous borrow 1 = 1 (borrow 1)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

11
=
(1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
--------------------------------------
 54       00110110        32+16+4+2
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

11
=
(1)1

0-1 = 1 (borrow 1) ->
1 (borrow 1) - previous borrow 1 = 0 (borrow 1)

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
-------------------------------------
 54       00110110        32+16+4+2
```

1-1 = 0 ->
0 - previous borrow 1 = 1 (borrow 1)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

11
=
(1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
----------------------------------------
 54       00110110        32+16+4+2
```

0 - 0 = 0 ->
0 - previous borrow 1 = 1 (borrow 1)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

11
=
(1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
----------^-------------------------
 54       00110110        32+16+4+2
```

0 - 1= 1 (borrow 1) ->
1 (borrow 1) - previous borrow 1 = 0 (borrow 1)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

11
=
(1)1

Examples: operations with positive integers

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
----------------------------------------
 54       00110110        32+16+4+2
```

1 -0 = 1 ->
1 - previous borrow 1 = 0

00110110 (binary) = 54 (decimal)

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

11
=
(1)1

Examples: operations with positive integers

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0  (1)
0 + 1 = 1
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1  (1)
```

Note: if it exceed the range, the remainder bit is lost (overflow) -> arithmetic module 2^N

```
223 +     11011111      128+64+16+8+4+2+1
 54       00110110      32+16+4+2
-------------------------------------------
          00010101      16+4+1   (223+54-256=21)
```

1 + 0 = 1 ->
1 + (p.carry 1) =
0 (*carry 1*)

(carry 1)

8 bits -> [0, 255] .. (2^n-1)

00010101 -> 21
100010101 → 277

Examples: operations with positive integers (recap)

```
 95 +     01011111        64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------
149       10010101        128+16+4+1
```

```
149 -     10010101        128+16+4+1
095       01011111        64+16+8+4+2+1
-------------------------------------
 54       00110110        32+16+4+2
```

```
  0 + 0 = 0
  1 + 0 = 1
  1 + 1 = 0  (1)
  0 + 1 = 1
```

```
  0 - 0 = 0
  1 - 0 = 1
  1 - 1 = 0
  0 - 1 = 1  (1)
```

Note: if it exceed the range, the remainder bit is lost (overflow) -> arithmetic module 2^N

```
223 +     11011111        128+64+16+8+4+2+1
 54       00110110        32+16+4+2
-------------------------------------------
 21       00010101        16+4+1   (223+54-256=21)
```

Integers with sign (signed)

- Signed integers can be represented by various encodings
- The most used coding are:
  - encoding sign-value
  - encoding 2-complement

Encoding sign-value

- The first bit represents the sign, the remaining bits represent the value
  - 1 represents a negative number
  - 0 represents a positive number
  - E.g., (8 bits): 10001001 represents -9

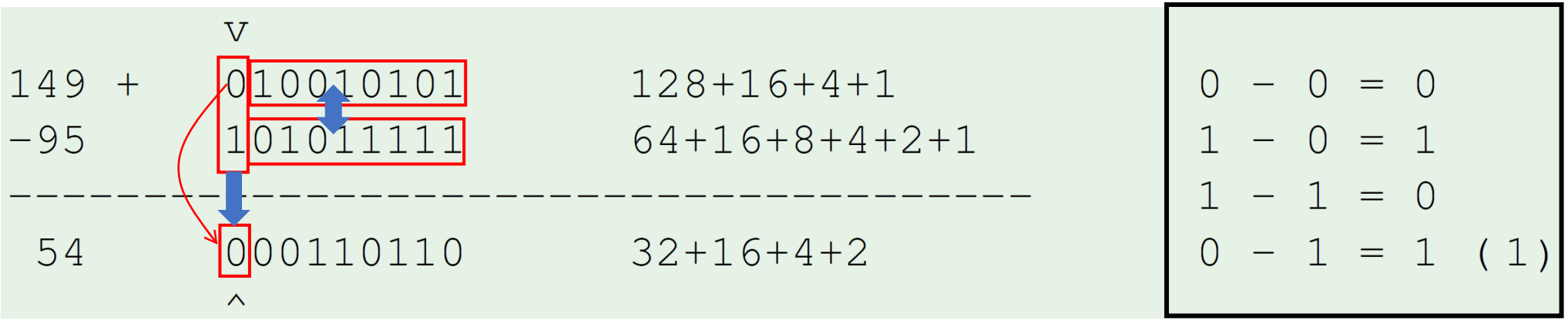
- It implies a double representations of the zero:
  - 00000000 and 10000000
- The numbers represented are in the range $[-2^{n-1} + 1, 2^{n-1}-1]$
  - E.g., with 32 bits [-2147483649; 2147483647]
- It is not so used in practice

Note

- It is necessary to use 2 different algorithms for the sum, depending on the sign: it could agree or disagree

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated



Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     01001010 1        128+16+4+1
-95       10101111 1        64+16+8+4+2+1
-------------------------------------------
 54       00011011 0        32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

1-1 = 0

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     01001010 1       128+16+4+1
-95       10101111 1       64+16+8+4+2+1
-------------------------------------------
 54       00011010 0       32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

0 - 1 = 1 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     01001010 1       128+16+4+1
-95       10101111 1       64+16+8+4+2+1
-------------------------------------------
 54       00011011 0       32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

1 - 1 = 0 →
0 - previous borrow 1 = 1 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     01001|0|101        128+16+4+1
-95       10101|1|111        64+16+8+4+2+1
-------------------------------------------
 54       00011|0|110        32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

0 - 1 = 1 (borrow 1) →
1 (borrow 1) - previous borrow 1 = 0 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     01001 0101        128+16+4+1
-95       10101 1111        64+16+8+4+2+1
-------------------------------------------
 54       00011 0110        32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

1 - 1 = 0 →
0 - previous borrow 1 = 1 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     0100 10101       128+16+4+1
-95       1010 11111       64+16+8+4+2+1
-------------------------------------------
 54       0001 10110       32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

0 - 0 = 0 →
0 - previous borrow 1 = 1 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     0100010101        128+16+4+1
-95       0010111111        64+16+8+4+2+1
-----------------------------------------
 54       0000110110        32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

0 - 1 = 1 (borrow 1) →
1 (borrow 1) - previous borrow 1 = 0 (borrow 1)

Examples: operations with sign-value

If the sign of the numbers is different, the greater of the two is established in absolute value, and then the sum or difference is calculated

```
          v
149 +     010010101        128+16+4+1
-95       101011111        64+16+8+4+2+1
----------------------------------------
 54       000110110        32+16+4+2
          ^
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

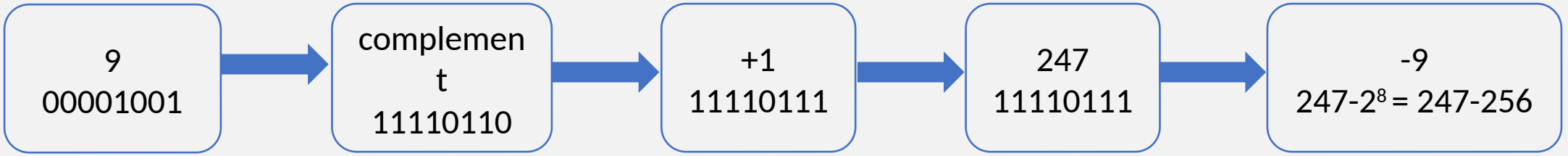
1 - 0 = 1 →
1 - previous borrow 1 = 0

00110110 (binary) = 54 (decimal)

Encoding 2-complement (*example 1*)

- The most significative bit represent the sign information. A negative number is obtained by calculating the complement of the positive number (zeros and ones are inverted) and then adding 1
- It implies a unique representation of the zero: 00000000
- The numbers represented are in the range $[-2^{n-1}, 2^{n-1}-1]$
  - E.g., (with 32 bits) [-2147483648; 2147483647]
  - E.g., (with 8 bits) [-128; 127]
- It is a cyclic representation: $-X$ in a 2-complement is written as 2^n-X in the representation without the sign (unsigned)

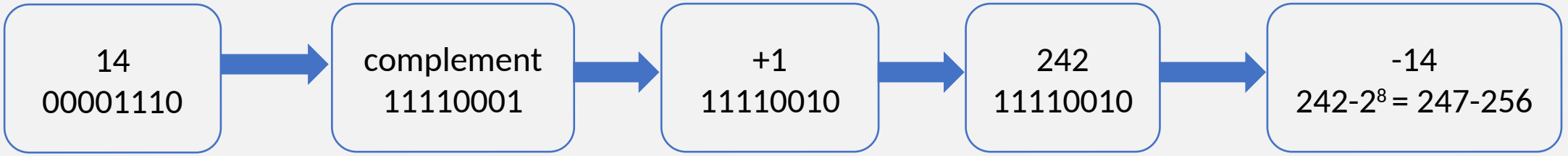
Example: 9 and -9 in 2-complement (8-bit representation)



Encoding 2-complement (*example 2*)

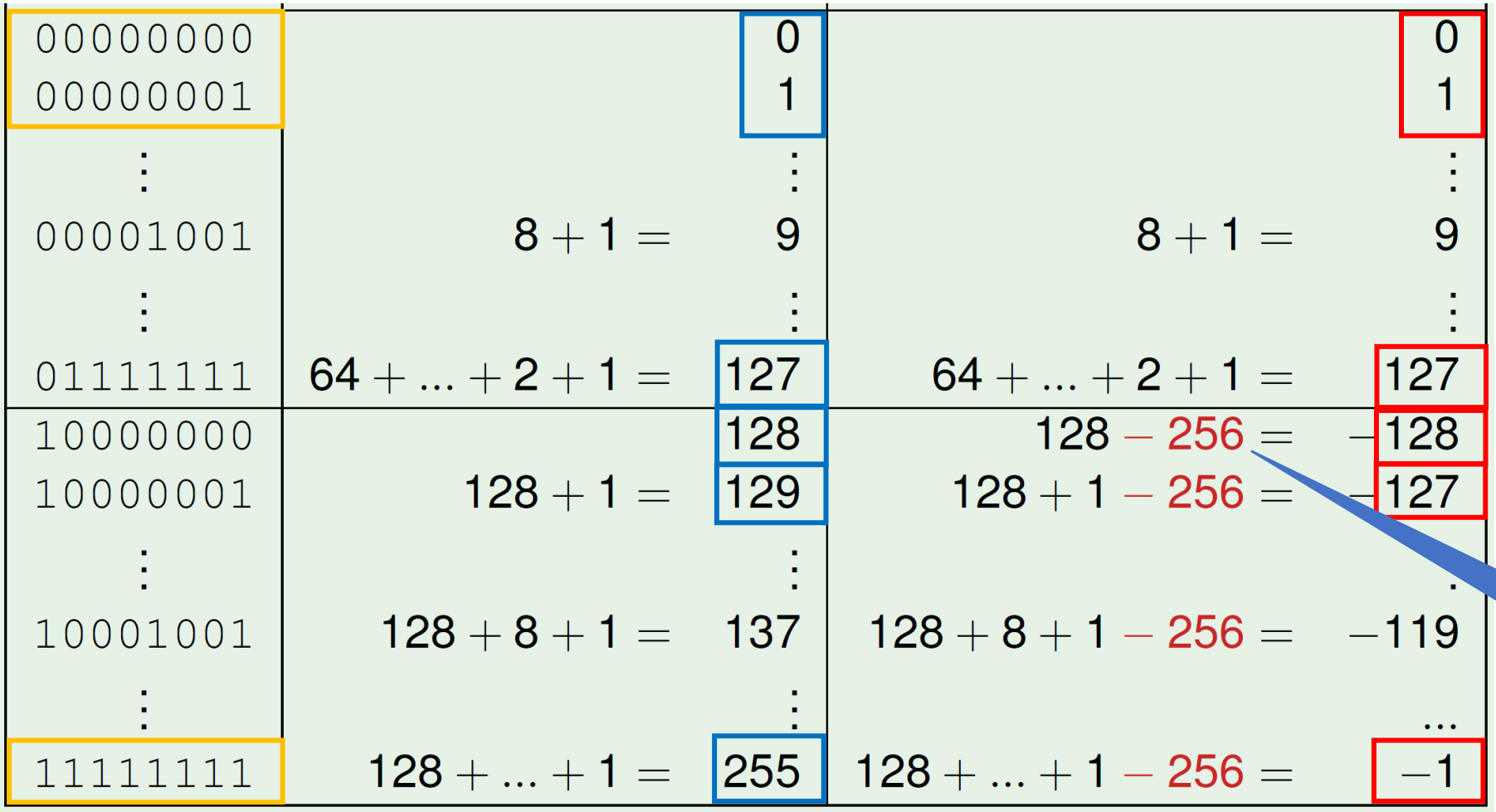
- The most significative bit represent the sign information. A negative number is obtained by calculating the complement of the positive number (zeros and ones are inverted) and then adding 1
- It implies a unique representation of the zero: 00000000
- The numbers represented are in the range $[-2^{n-1}, 2^{n-1}-1]$
  - E.g., (with 32 bits) [-2147483648; 2147483647]
  - E.g., (with 8 bits) [-128; 127]
- It is a cyclic representation: $-X$ in a 2-complement is written as 2^n-X in the representation without the sign (unsigned)
- **Applying 2-complement twice gets you the number you started from: X = f(f(X))**

Example: 14 and -14 in 2-complement (8-bit representation)



11110111 in the representation without the sign is 242 (= 2^8-14=256-14)

Integer with sign vs. 2-complement



Examples: Operations in 2-complement

```
    −9  +  11110111
     9     00001001
=    0     00000000
```

(9 → 00001001)

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0  (1)
0 + 1 = 1
```

1 + 1 = 0 (1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
0 + 1 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)

00000000

- 2 complement of the negative number(s) → becomes a sum
- If the result is negative, then 2-complement of the result makes it simpler to understand which is the result

Examples: Operations in 2-complement

```
    -9 + 11110111
     8   00001000
=   -1   11111111    => 11111110 => 00000001
```

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0 (1)
0 + 1 = 1
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

1 + 0 = 1(0)
1 + 0 = 1 + (0) → 1(0)
1 + 0 = 1 + (0) → 1(0)
0 + 1 = 1 + (0) → 1(0)
1 + 0 = 1 + (0) → 1(0)
1 + 0 = 1 + (0) → 1(0)
1 + 0 = 1 + (0) → 1(0)
1 + 0 = 1 + (0) → 1(0)

11111111 —complement→ 00000000 —+1→ 00000001

- 2 complement of the negative number(s) → becomes a sum
- If the result is negative, then 2-complement of the result makes it simpler to understand which is the result

Examples: Operations in 2-complement

```
   -9 + ➡ 11110111
   10     00001010
=   1     00000001
```

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0 (1)
0 + 1 = 1
```

1 + 0 = 1(0)
1 + 1 = 0(1) + (0) → 0(1)
1 + 0 = 1 + (1) → 0(1)
0 + 1 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)
1 + 0 = 1 + (1) → 0(1)

00000001

- 2 complement of the negative number(s) → becomes a sum
- If the result is negative, then 2-complement of the result makes it simpler to understand which is the result

Examples: Operations in 2-complement

```
    -9 + → 11110111
    -5   → 11111011
=   -14      11110010    => 11110001 => 00001110
```

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0 (1)
0 + 1 = 1
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

1 + 1 = → 0(1)
1 + 1 = 0(1) + (1) → 1(1)
1 + 0 = 1 + (1) → 0(1)
0 + 1 = 1 + (1) → 0(1)
1 + 1 = 0(1) + (1) → 1(1)
1 + 1 = 0(1) + (1) → 1(1)
1 + 1 = 0(1) + (1) → 1(1)
1 + 1 = 0(1) + (1) → 1(1)

11110010 → (complement) 00001101 → (+1) 00001110

- 2 complement of the negative number(s) → becomes a sum
- If the result is negative, then 2-complement of the result makes it simpler to understand which is the result

Examples: Operations in 2-complement (recap)

```
    -9 +     11110111
     9       00001001
=    0       00000000

    -9 +     11110111
     8       00001000
=   -1       11111111     => 11111110 => 00000001

    -9 +     11110111
    10       00001010
=    1       00000001

    -9 +     11110111
    -5       11111011
=   -14      11110010    => 11110001 => 00001110
```

```
0 + 0 = 0
1 + 0 = 1
1 + 1 = 0 (1)
0 + 1 = 1
```

```
0 - 0 = 0
1 - 0 = 1
1 - 1 = 0
0 - 1 = 1 (1)
```

- 2complement of the negative number
- If the greatest one is negative, then 2complement of the result (no sign)

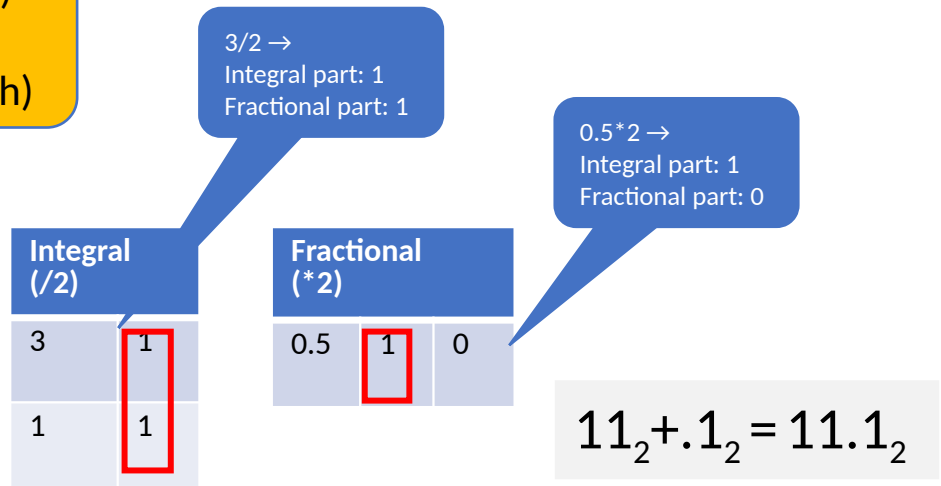
Decimal to Binary, with decimal separator

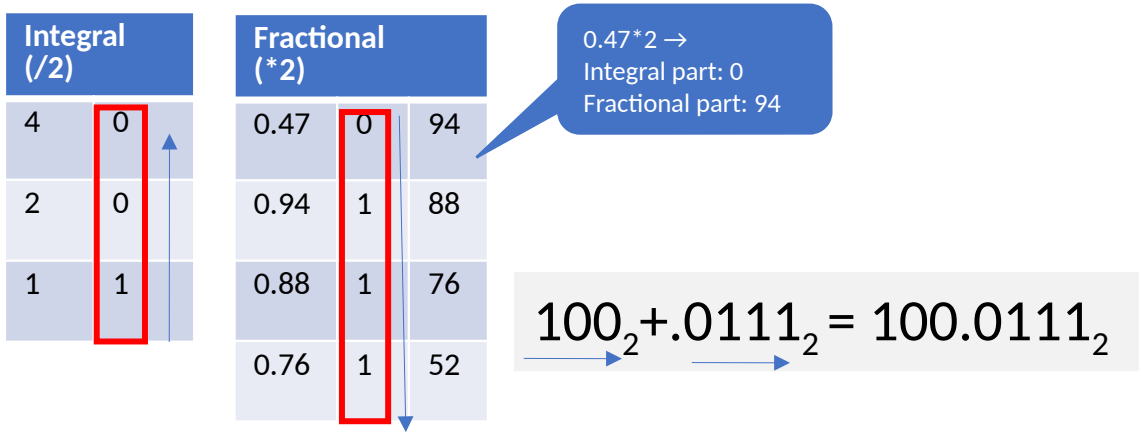
Decimal numbers can have the decimal separator (comma or dot)

1. Take the integral part (as digit) and convert it in binary (division by 2 etc.)
2. Multiply the fractional part *2
3. Repeat using the fractional (stop if fractional is 0 or when bits are enough)

Conversion, e.g.,

- To convert 3.5_{10} in binary with comma
  - Integral part: $3_{10} \rightarrow 011_2$
  - Fractional part: $0.5_{10} \rightarrow 0.1_2$
  - Result: 11.1_2
- To convert 4.47_{10} in binary with comma
  - Integral part: $4_{10} \rightarrow 100_2$
  - Fractional part: $0.47_{10} \rightarrow 0.0111_2$
  - Result: 100.0111_2





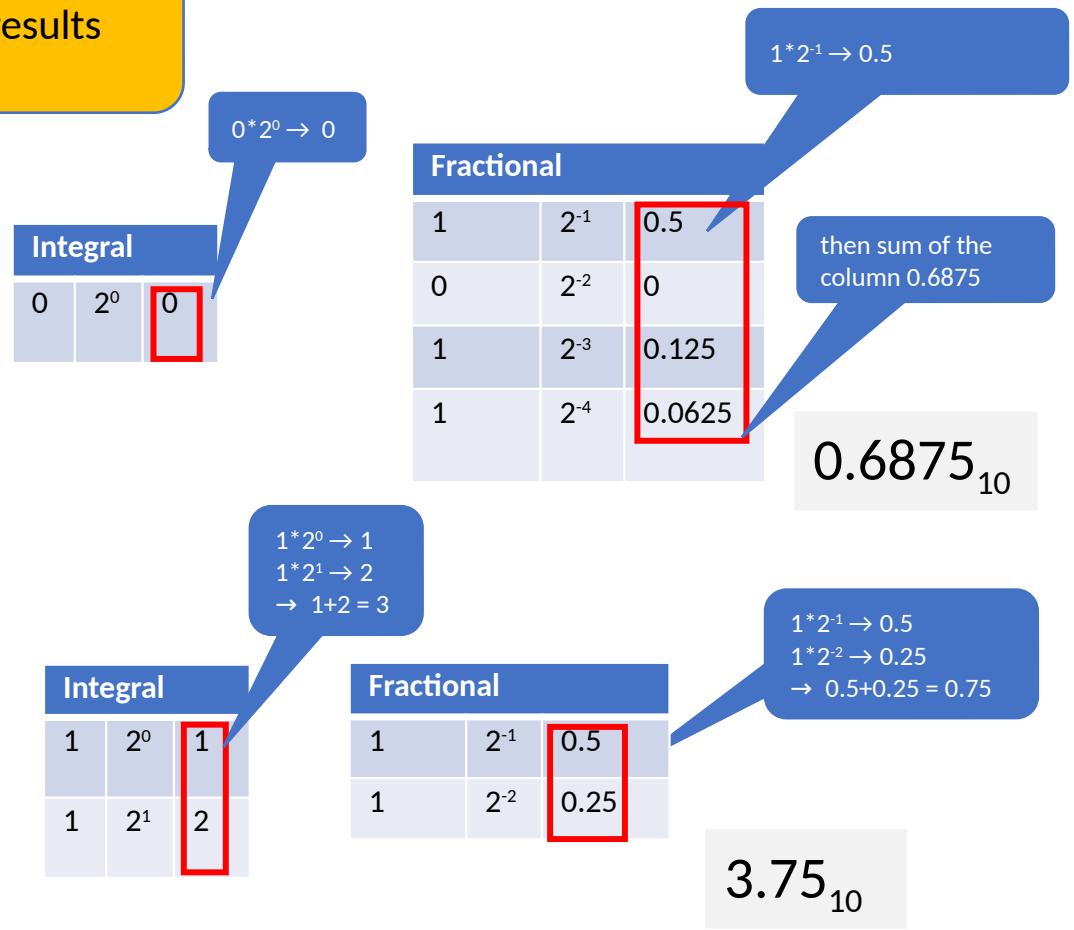
Binary with separator to decimal with separator

Convert binary with comma to decimal with separator (comma or dot)

Multiply each digit (0 or 1) of the integral part $*2^x$ and sum the results
Multiply each digit (0 or 1) of the fractional part $*2^{-x}$ (=) and sum the results
where *x* is the position of the digit

Conversion, e.g.,

- To convert 0.1011_2 in decimal with comma
  - Integral part: $0_2 \rightarrow 0_{10}$
  - Fractional part: $0.1011_2 \rightarrow 0.6875_{10}$
  - Result: 0.6875_{10}
- To convert 11.11_2 in decimal with comma
  - Integral part: $11_2 \rightarrow 3_{10}$
  - Fractional part: $0.11_2 \rightarrow 0.75_{10}$
  - Result: 3.75_{10}



Real numbers

Floating-point representation (IEEE 745)

Syntax:

$$(-1)^s * m * 2^{e-o}$$

s is sign ({-1,+1}), m is the mantissa, e is exponential, o is offset

- The mantissa is expressed in decimal binary encoding
  - E.g., ".1001… " → "1*+ 0* + 0 + + 1 * + …"
  - Note: (normal representation) the first bit is assumed to be 1 (e.g., "000100001…" is "1.000100001…")
  - It requires special algos for operations

Real numbers

$$(-1)^s * m * 2^{e-o},$$

s is sign ({-1,+1}), m is the mantissa, e is exponential, o is offset

Format (bit)	Sign (bit)	Exp (bit)	Mantissa (bit)	offset	Range	Precision
32	1	8	23	2^7-1 (127)	$\approx[10^{-38}, 10^{38}]$	≈ 6 decimal digits
64	1	11	52	$2^{10}-1$ (1023)	$\approx[10^{-308}, 10^{308}]$	≈ 16 decimal digits
128	1	15	112	$2^{14}-1$ (16383)	$\approx[10^{-4932}, 10^{4932}]$	≈ 34 decimal digits

Single precision

Double precision

Why, why, why???

- Why the offset?
  - We need to represent both small and big numbers. However, a positive exponent is good for big numbers only

    → The offset allows to represent also small numbers.

- Why not 2-complement for the exponent, then?
  - By shifting, we keep the same order and the smallest exponent is smaller the the highest exponent (this does not happen with 2-complement, see slide 49)

    → Order comparison can be performed by simply comparing the corresponding bit strings lexicographically, or equivalently, by interpreting those bit strings as unsigned integers and comparing those integers

Conversion IEEE745 (1)

- Convert 0.1875_{10} in floating-point with 32 bits (single precision)

Integral (/2)	
0	0

Fractional (*2)		
0.1875	0	375
0.3750	0	75
0.75	1	5
0.5	1	0

0.1875*2 ->
Integral part: 0
Fractional part: 375

0.0011_2 → *scientific notation* → $1.1_2 * 2^{-3}$

- $(-1)^s * (1+\text{Mantissa}) * 2^{\text{Exponent}-127}$
  - $(-1)^s = 1 \rightarrow s = 0$
  - Mantissa - 1 => 1.1 – 1 => 0.1
  - (Exponent – Offset) => (-3 + 127) = 124 (bin: 01111100)

S|Exp. 8 bit | Mantissa 23 bit
0|01111100|10000000000000000000000

Conversion IEEE745 (2)

- Convert -10.75_{10} in floating-point with 32 bits (single precision)

Integral (/2)	
10	0
5	1
2	0
1	1

Fractional (*2)		
0.75	1	5
0.5	1	0

-1010.11_2 → *scientific notation* → $-1.01011_2 * 2^3$

- $(-1)^s * (1+\text{Mantissa}) * 2^{\text{Exponent}-127}$
  - $(-1)^s = -1 \rightarrow s = 1$
  - (1+Mantissa) = 1.01011 → Mantissa = 1.01011 - 1 = 0.01011
  - (Exponent - 127) = 3 → Exponent = 130 (bin: 10000010)

S|Exp. 8 bit |Mantissa 23 bit
1|10000010|01011000000000000000000

Special Representations

- **Zero:**
  - Everything zeros but the sign bit (0 → positive, 1 → negative)
  - Good for 1 / +0 and 1 / -0

- **Infinity:**
  - Exponent: all 1s
  - Mantissa: all 0s
  - Sign for positive/negative infinity

- **NaN:**
  - Exponent: all 1s, Mantissa: != 0, Sign bit
  - How many NaNs?

Hex to Binary conversion

- The **hexadecimal system (hex)**, uses 16 symbols, base-16
  - It uses: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

1. Write each hex digit as its binary equivalent
2. Take 4 of the binary digits (truncate, if greater than 4 digits, or fill with 0, if less than 4 digits)
3. Concatenate the 4-digit binary

E.g., $0x3AB2_{16}$

3 → 011 → 0011
A → 1010 → 1010
B → 1011 → 1011
2 → 10 → 0010
→ 0011101010110010

E.g., $0x427d0000_{16}$

4 → 0100
2 → 0010
7 → 0111
D → 1101
0 → 0000
0 → 0000
0 → 0000
0 → 0000
→ 01000010011111010000000000000000

Dec	Hex	Bin
0	0	0
1	1	1
2	2	10
…	…	…
8	8	1000
9	9	1001
10	A	1010
11	B	1011
…	…	…
15	F	1111
16	10	10000
17	11	10001
18	12	10010
…	…	…

Binary to Hex conversion

1. Starting from the least significant bit (right), divide in groups of 4 digits
2. Convert each group of 4 digits to the hex corresponding one
3. Concatenate the result

E.g., 0011101010110010

0011 1010 1011 0010

0010 -> 2
1011 -> B
1010 -> A
0011 -> 3
-> $3AB2_{16}$

E.g., 111011.111

111011.111

1110 -> E
1011 -> B
0011 -> 3
-> $3B.E_{16}$

E.g., 1111011101110010

1111 0111 0111 0010

0010 -> 2
0111 -> 7
0111 -> 7
1111 -> F
0001 -> 1
-> $1F772_{16}$

Dec	Hex	Bin
0	0	0
1	1	1
2	2	10
…	…	…
8	8	1000
9	9	1001
10	A	1010
11	B	1011
…	…	…
15	F	1111
16	10	10000
17	11	10001
18	12	10010
…	…	…

Conversion IEEE745 (3)

- Convert $0x427d0000_{16}$ in floating-point decimal

1. Hexadecimal to Binary

   $0x427d0000_{16} = 0100\ 0010\ 0111\ 1101\ 0000\ 0000\ 0000\ 0000_2$

   $0 \mid 10000100 \mid 11111010000000000000000_2$

2. $N = (-1)^s * (1+Mantissa) * 2^{Exponent-127}$
   - $s = 0$
   - $Exponent = 10000100 = 2^7 + 2^2 = 132$

   -> $Exponent - 127 = 132 - 127 = 5$

   - $Mantissa = 11111010000000000000000 = 2^{-1} + 2^{-2} + 2^{-3} + 2^{-4} + 2^{-5} + 2^{-7} = 0.9765625$

   $N = (-1)^0 * (1+0.9765625) * 2^5 = 1 * 1.9765625 * 32 = 63.25$